



Bilgisayar Programlama

Bellek Yönetimi II: Dinamik Bellek ve 2D Diziler

İçindekiler

1. Giriş: Statik vs. Dinamik Bellek

- Heap ve Stack Alanları, Neden Dinamik Bellek?

2. `malloc()` Fonksiyonu

- Temel Tahsis, `sizeof` Kullanımı ve `NULL` Kontrolü

3. `calloc()` Fonksiyonu

- Başlatılmış (Sıfırlanmış) Bellek ve `malloc` ile Farkları

4. `realloc()` Fonksiyonu

- Güvenli Boyut Değiştirme ve `DAQ` Senaryoları

5. `free()` ve Kritik Bellek Hataları

- Sızıntılar, Dangling Pointer ve Double Free

İçindekiler (devam)

6. Double Pointer (İşaretçi İşaretçisi)

- Katmanlı Adresleme ve Mantıksal Yapı

7. 2 Boyutlu Dinamik Diziler

- Double Pointer vs. Tek Blok Yaklaşımı

8. Hata Ayıklama Araçları ve İyi Pratikler

- Valgrind, AddressSanitizer ve Güvenli Kodlama

9. Laboratuvar Çalışmaları ve Çözümler



BÖLÜM 1

Giriş: Statik vs. Dinamik Bellek

1.1. Neden Dinamik Bellek?

Hafta 8'de açık kalan soruyu hatırlayalım: *"Bir programın çalışma zamanında kaç sensör verisi geleceğini önceden bilmiyorsak, sabit boyutlu diziler yeterli olur mu?"*

Statik Bellek (Diziler):

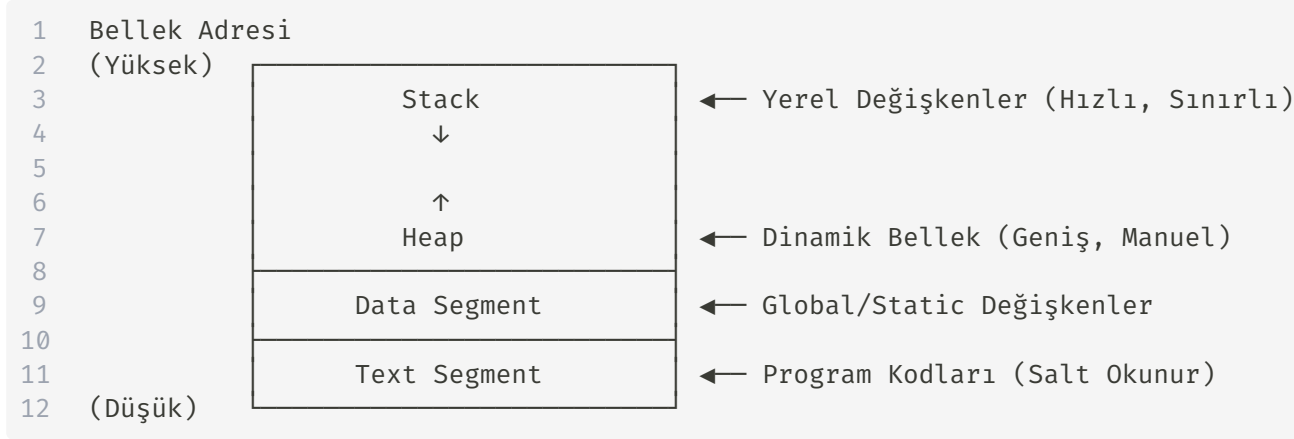
- Derleme zamanında boyutu bilinir (örn: `int veriler[100];`).
- Stack (Yığın) üzerinde yaşar.
- Kapsam (scope) dışına çıkıldığında otomatik silinir.

Dinamik Bellek:

- Çalışma zamanında (runtime) programcının isteği üzerine ayrılır.
- Heap (Öbek) alanında yaşar.
- Programcı tarafından manuel yönetilir (ayrılır ve iade edilir).

1.2. Bellek Alanları (Memory Segments)

Gömülü sistemlerde (MCU) bellek alanlarını tanımak hayati önem taşır:



⚠ **Mekatronik Notu:** MCU'larda RAM kapasitesi çok sınırlıdır (birkaç KB). Stack taşması (Overflow) cihazın aniden resetlenmesine sebep olur!

1.3. Stack vs. Heap Karşılaştırması

Özellik	Stack (Yığın)	Heap (Öbek)
Yönetim	Otomatik (Derleyici)	Manuel (<code>malloc</code> / <code>free</code>)
Boyut	Sınırlı (MB/KB seviyesi)	Geniş (Sistem RAM'i kadar)
Hız	Çok hızlı	Daha yavaş
Yaşam Süresi	Fonksiyon bitince silinir	<code>free</code> çağrılana kadar yaşar
Riskler	Stack Overflow	Bellek Sızıntısı, Parçalanma

BÖLÜM 2

`malloc()` Fonksiyonu

2.1. malloc() Nedir?

C'de dinamik bellek ayırmak için standart kütüphane (`stdlib.h`) kullanılır.

- Prototip: `void* malloc(size_t size);`
- İşlevi: `size` byte kadar ham (başlatılmamış) bellek alanı ayırır.
- Dönüş Değeri: Başarılıysa adres (`void*`), başarısızsa `NULL` döndürür.

⚠️ **Önemli:** Ayrılan bellek içeriği tanımsızdır (çöp veri içerebilir). Kullanmadan önce mutlaka değer atamalısınız.

2.2. Generic Pointer: `void*` Kavramı

Dinamik bellek fonksiyonları (`malloc` , `calloc` , `realloc`) her zaman `void*` tipinde bir adres döndürür.

- Nedir? Herhangi bir veri tipini göstermeyen, sadece "bellekte bir adres" belirten genel (generic) işaretçidir.
- Neden Kullanılır? `malloc` fonksiyonu, sizin bu bellekte bir `int` mi, `float` mı yoksa bir `struct` mı saklayacağınızı önceden bilemez. Bu yüzden "ham bir adres" verir.

C vs C++ Farkı:

- C: `void*` her tipe otomatik dönüşebilir. `(int*)` yazmak teknik olarak zorunlu değildir ama iyi pratiktir.
- C++: Tip dönüşümü (Cast) zorunludur. Kodun her iki dilde de çalışması için cast yapmak en güvenli yoldur.

2.3. sizeof Operatörünün Önemi

Platform bağımsız ve güvenli kod yazmak için `sizeof` şarttır. `malloc` ile byte cinsinden boyut isteriz, ancak veri tiplerinin boyutu donanıma göre değişebilir.

✓ İyi / Doğru Kullanım:

```
1 // n adet integer için taşınabilir bellek ayırma
2 int *ptr = (int*) malloc(n * sizeof(int));
```

× Kötü / Hatalı Kullanım:

```
1 // Sadece int=4 byte olan PC'de çalışır, 8-bit MCU'da patlar!
2 int *ptr = (int*) malloc(n * 4);
```

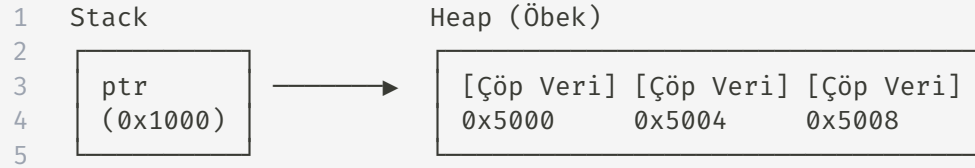
2.4. NULL Kontrolü (Zorunluluk!)

`malloc` bellek bulamazsa `NULL` döner. Kontrol etmeden bu adrese yazmaya çalışmak Segmentation Fault (Çökme) ile sonuçlanır.

```
1  int *arr = (int*) malloc(100 * sizeof(int));
2
3  if (arr == NULL) {
4      fprintf(stderr, "Kritik Hata: Bellek ayırma basarisiz!\n");
5      // Sistemi güvenli duruma al (Örn: Robot motorlarını durdur)
6      return 1;
7  }
8
9  // Artık arr güvenli kullanılabilir...
```

2.5. İzleme Tablosu: malloc Sonrası Bellek

```
int *ptr = (int*) malloc(3 * sizeof(int));
```

 işlemi sonrası:

Pointer (`ptr`), Heap'te yeni ayrılan 12 byte'lık (3 x 4 byte) bloğun başlangıç adresi olan `0x5000` değerini tutar.



BÖLÜM 3

`calloc()` Fonksiyonu

3.1. `calloc()` — Sıfırlanmış Bellek

- Prototip: `void* calloc(size_t num, size_t size);`
- İşlevi: `num` adet, her biri `size` byte olan bellek bloğu ayırır.
- ⚠ Kritik Fark: `calloc` , ayırdığı belleğin tüm bitlerini sıfırlar (`0` yapar).

Mekatronikteki Yeri: PID kontrolör hata dizileri veya dijital filtre tamponları gibi başlangıç değerinin sıfır olması kritik olan yapılarda `calloc` tercih edilir. Aksi halde çöp değerler motor titreşimine yol açabilir.

3.2. malloc vs calloc Karşılaştırması

Özellik	<code>malloc(n * sizeof(int))</code>	<code>calloc(n, sizeof(int))</code>
Argüman	1 (Toplam byte)	2 (Adet, Boyut)
İlk İçerik	Çöp Veri (Garbage)	Tamamen Sıfırlanmış (0)
Performans	Biraz daha hızlı	Sıfırlama nedeniyle yavaş
Kullanım Yeri	Hemen üzerine yazacaksınız	Başlangıçta 0 olması şartsa

3.3. Örnek: Motor PWM Kontrol Dizisi

```
1  int motor_sayisi = 6; // 6 eksenli robot kolu
2  int *pwm_degerler = (int*) calloc(motor_sayisi, sizeof(int));
3
4  if (pwm_degerler == NULL) return 1;
5
6  // Tüm motorlar güvenli başlangıçta 0 (durdurulmuş)
7  for (int i = 0; i < motor_sayisi; i++) {
8      printf("Motor %d: %d\n", i, pwm_degerler[i]); // Hepsi 0 çıkar
9  }
10
11 free(pwm_degerler);
```

BÖLÜM 4

`realloc()` Fonksiyonu

4.1. realloc() Nedir?

- Prototip: `void* realloc(void* ptr, size_t new_size);`
- İşlevi: Ayrılmış bir bellek bloğun boyutunu çalışma zamanında büyütür veya küçültür.

Davranış Senaryoları:

1. Büyütme: Bloğu büyütür. Eski veriler korunur. Yeterli yer yoksa belleği başka bir yere taşır, eskiyi siler.
2. Küçültme: Fazla kısmı serbest bırakır.
3. Başarısızlık: `NULL` döner ancak eski blok hala geçerlidir.

4.2. Güvenli `realloc` Kullanımı

⚠ Kritik Hata: Dönüş değerini doğrudan aynı pointer'a atamak! Başarısızlık durumunda orijinal adresi kaybedersiniz (Bellek Sızıntısı).

× Yanlış:

```
1 ptr = (int*) realloc(ptr, 100 * sizeof(int)); // NULL dönerse veri çöp olur!
```

✓ Doğru (Geçici Pointer ile):

```
1 int *temp = (int*) realloc(ptr, 100 * sizeof(int));
2 if (temp != NULL) {
3     ptr = temp; // Başarılıysa adresi güncelle
4 } else {
5     // Başarısız! ptr hala eski adreste hayatta, sızıntı yok.
6 }
```

4.3. Örnek: Dinamik Sensör Veri Toplama (DAQ)

```
1  int kapasite = 3;
2  int *veriler = (int*) malloc(kapasite * sizeof(int));
3
4  // ... ilk 3 sensör verisi doldu ...
5
6  // Yeni modül eklendi, kapasiteyi 5'e çıkaralım
7  int *temp = (int*) realloc(veriler, 5 * sizeof(int));
8  if (temp != NULL) {
9      veriler = temp;
10     veriler[3] = 40; veriler[4] = 50; // Yeni alanları doldur
11 }
12
13 free(veriler);
```

BÖLÜM 5

`free( )` ve Kritik Bellek Hataları

5.1. free() Kullanımı

Dinamik ayrılan bellek sisteme iade edilmelidir. Aksi halde "Bellek Sızıntısı" oluşur.

- Kural: Her `malloc/calloc` için bir `free` yazılmalıdır.
- Güvenlik: `free(NULL)` hiçbir şey yapmaz, güvenlidir.

✓ İyi Pratik (Dangling Pointer Önlemi):

```
1 free(ptr);  
2 ptr = NULL; // Artık ptr geçersiz bir adresi göstermez
```

5.2. Yaygın Bellek Hataları

1. × Memory Leak (Sızıntı): `free()` yapılmaması. Uzun süreli robotik döngülerde RAM dolar ve sistem çöker.
2. × Dangling Pointer (Sallanan İşaretçi): Serbest bırakılmış bir adrese erişmeye devam etmek.
3. × Double Free: Aynı belleği iki kez `free` etmek. Heap yapısını bozar, anında çökme (Abort) yaratır.
4. × Buffer Overflow: Ayrılan alanın dışına yazmak. Yandaki verileri ezer.
5. × NULL Dereference: `NULL` olan bir pointer'ı kullanmaya çalışmak (Segfault).

BÖLÜM 6

Double Pointer (İşaretçi İşaretçisi)

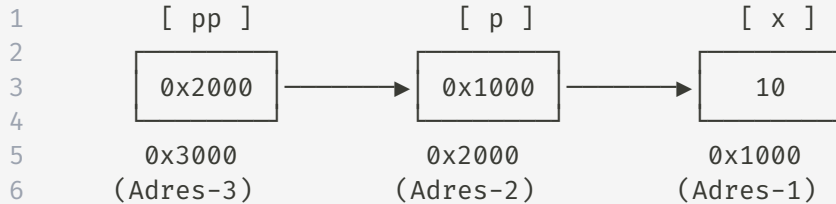
6.1. Double Pointer (İşaretçi İşaretçisi)

Double Pointer (`**`), bir işaretçinin bellekteki adresini saklayan özel bir işaretçidir. Bellekteki bir veriye dolaylı yoldan (iki kademeli) erişim sağlar.

Temel Yapı:

```
1  int x = 10;      // Veri
2  int *p = &x;     // p, x'in adresini tutar (1. seviye)
3  int **pp = &p;   // pp, p işaretçisinin adresini tutar (2. seviye)
```

Bellek Organizasyonu:



6.2. Erişim Seviyeleri (Dereferencing)

Double pointer (`pp`) üzerinden verilere farklı seviyelerde erişilebilir. Her bir yıldız (`*`) işlemi, bellekte bir adım ileri gitmeyi temsil eder.

İfade	Döndürdüğü Değer	Teknik Karşılığı
<code>pp</code>	<code>0x3000</code>	İşaretçinin (<code>p</code>) kendi adresi
<code>*pp</code>	<code>0x2000</code>	İşaretçinin içindeki değer (<code>x</code> 'in adresi)
<code>**pp</code>	<code>10</code>	Hedeflenen asıl veri (<code>x</code>)

📌 Not: `*pp` ifadesi teknik olarak `p` işaretçisine eşdeğerdir. Bu sayede fonksiyonlar içinde asıl işaretçinin adresi güncellenebilir.

6.3. Neden İhtiyaç Duyarız? (Fonksiyon Parametreleri)

Bir fonksiyonun içinden, dışarıdaki bir pointer'ın kendisini (gösterdiği adresi) kalıcı olarak değiştirmek istiyorsak double pointer kullanmalıyız.

× Hatalı (Değişiklik Fonksiyon Dışına Yansımaz):

```
1 void bellekAyir(int *ptr) {  
2     ptr = (int*) malloc(sizeof(int)); // Sadece yerel kopya değişir  
3 }
```

✓ Doğru (Asıl Pointer Güncellenir):

```
1 void bellekAyir(int **pptr) {  
2     *pptr = (int*) malloc(sizeof(int)); // Dışarıdaki asıl pointer güncellenir  
3 }
```

⚠ Mekantronik Notu: Robotik sistemlerde sensör listelerini veya dinamik kontrolör parametrelerini yöneten "Yönetici Fonksiyonlar" (Manager Functions) bu yapıyı yoğun olarak kullanır.

6.4. Daha İleri Seviye: Triple Pointer (`***`) ve Ötesi

C dilinde teorik olarak sonsuz sayıda işaretçi seviyesi (`***ptr` , `****ptr` ...) tanımlanabilir. Ancak pratikte 2. seviye (`**`) yaygın, 3. seviye ise nadiren (çok boyutlu dinamik dizilerde) kullanılır.

Kritik Kural: Her bir yıldız (`*`), bir seviye "içeri" girmeyi (dereference) temsil eder.

- `int x` : Veri
- `int *p = &x` : Verinin adresi
- `int **pp = &p` : Adresin adresi
- `int ***ppp = &pp` : Adresin adresinin adresi



BÖLÜM 7

2 Boyutlu Dinamik Diziler

7.1. Yöntem 1: Double Pointer (Satır Bazlı)

Önce "satırların adreslerini" tutacak bir pointer dizisi, sonra her satır için sütunlar ayrılır.

```
1  int satir = 3, sutun = 4;
2  int **mat = (int**) malloc(satir * sizeof(int*));
3
4  for (int i = 0; i < satir; i++) {
5      mat[i] = (int*) malloc(sutun * sizeof(int));
6  }
7
8  mat[1][2] = 45; // Doğal erişim
```

⚠ Dezavantaj: Satırlar bellekte dağınık (parçalı) yerleşebilir. Ön bellek (Cache) performansı düşüktür.

7.2. Yöntem 2: Tek Blok (Ardışık Bellek) ✓

Tüm matris tek bir uzun 1D dizi olarak ayrılır. Gömülü sistemlerde hız için bu tercih edilir.

```
1 int satir = 3, sutun = 4;
2 int *mat = (int*) malloc(satir * sutun * sizeof(int));
3
4 // Erişim Formülü: [satir * SUTUN_SAYISI + sütun]
5 mat[1 * sutun + 2] = 45;
```

✓ Avantaj: Tamamen ardışık bellek (Hızlı), tek `malloc` ve tek `free` .

7.3. Karşılaştırma Tablosu

Özellik	Double Pointer (<code>int**</code>)	Tek Blok (<code>int*</code>)
Erişim	<code>mat[i][j]</code> ✓	<code>mat[i * cols + j]</code>
Bellek Düzeni	Parçalı	Ardışık (Cache Dostu) ✓
Tahsis/İade	Döngü Gerektirir	Tek Satır ✓
Esneklik	Jagged Array (Değişken satır) ✓	Sabit sütun

BÖLÜM 8

Hata Ayıklama Araçları ve İyi Pratikler

8.1. Hata Ayıklama (Debugging) Araçları

Bellek hataları C dilinin en zor tespit edilen hatalarıdır. Sizi kurtaracak araçlar:

1. Valgrind (Memcheck):

- Linux/macOS altın standardıdır. Sızıntıları ve geçersiz erişimleri raporlar.
- *Kullanım:* `valgrind --leak-check=full ./program`

2. AddressSanitizer (ASan):

- Derleyiciye entegredir, çok hızlıdır.
- *Kullanım:* `gcc -fsanitize=address -g kod.c`

3. GDB / LLDB:

- Çökme anındaki bellek dökümünü (Memory Dump) incelemek için.

8.2. İyi Pratikler (Best Practices)

1. Eşleştirme: Yazdığın her `malloc` için bir `free` yaz.
2. NULL Kontrolü: Sonucu kontrol etmeden asla kullanma.
3. Dangling Önlemi: `free` sonrası pointer'ı `NULL` yap.
4. Taşınabilirlik: Boyut hesabını `sizeof(tip)` ile yap.
5. Stack Tercihi: Küçük ve sabit veriler için Stack her zaman daha güvenli ve hızlıdır.



BÖLÜM 9

Laboratuvar Çalışmaları ve Çözümler

9.1. Lab 1: Dinamik Sensör Dizisi

Görev: Kullanıcıdan sensör sayısını alan, rastgele değerler atayan, ortalamayı hesaplayıp belleği iade eden program.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4
5  int main() {
6      int n;
7      printf("Sensor sayisi: "); scanf("%d", &n);
8
9      float *sicakliklar = (float*) malloc(n * sizeof(float));
10     if (sicakliklar == NULL) return 1;
11
12     srand(time(NULL));
13     float toplam = 0;
14     for (int i = 0; i < n; i++) {
15         sicakliklar[i] = (rand() % 500) / 10.0;
16         toplam += sicakliklar[i];
17     }
18     printf("Ortalama: %.2f\n", toplam / n);
19     free(sicakliklar);
20 }
```

9.2. Lab 2: Dinamik Dizi Büyütme (realloc)

Görev: Başlangıçta 3 elemanlı dizi kurup, kullanıcı istedikçe `realloc` ile büyüten yapı.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main() {
5      int boyut = 3;
6      int *dizi = (int*) malloc(boyut * sizeof(int));
7      if (dizi == NULL) return 1;
8
9      char cevap;
10     while (1) {
11         printf("Yeni veri ekle? (E/H): "); scanf(" %c", &cevap);
12         if (cevap == 'H' || cevap == 'h') break;
13
14         int yeni_veri;
15         printf("Veri: "); scanf("%d", &yeni_veri);
16
17         boyut++;
18         int *temp = (int*) realloc(dizi, boyut * sizeof(int));
19         if (temp == NULL) { free(dizi); return 1; }
20         dizi = temp;
```

9.3. Lab 3: Matris Transpozu

Görev: $M \times N$ matris oluşturup transpozunu yeni bir dinamik matrise kaydeden program.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main() {
5      int M, N;
6      printf("M N girin: "); scanf("%d %d", &M, &N);
7
8      // Bellek Ayırma
9      int **mat = (int**) malloc(M * sizeof(int*));
10     for (int i = 0; i < M; i++) mat[i] = (int*) malloc(N * sizeof(int));
11
12     int **trans = (int**) malloc(N * sizeof(int*));
13     for (int i = 0; i < N; i++) trans[i] = (int*) malloc(M * sizeof(int));
14
15     // İşlem
16     for (int i = 0; i < M; i++)
17         for (int j = 0; j < N; j++) {
18             mat[i][j] = i + j; // Örnek veri
19             trans[j][i] = mat[i][j];
20     }
```